

A PROOFS OF THEOREMS FROM § 3

A.1 Lemma 3.9

For any relation $R \subseteq X \times Y$ the functions Δ_R and ∇_R are conjugate.

PROOF. Let $X' \in \mathbb{P}(X)$, $Y' \in \mathbb{P}(Y)$.

WTS: $\Delta_R(X') \cap Y' = \emptyset \implies X' \cap \nabla_R(Y') = \emptyset$

$\Delta_R(X') \cap Y' = \emptyset$

suppose

$\{y \in Y \mid \exists x \in X'. (x, y) \in R\} \cap Y' = \emptyset$

def of Δ_R

$\{y \in Y' \mid \exists x \in X'. (x, y) \in R\} = \emptyset$

def of $\cap$ (1)

$\forall y \in Y'. \forall x \in X'. (x, y) \notin R$

equivalent to (1) (2)

$\{x \in X' \mid \exists y \in Y'. (x, y) \in R\} = \emptyset$

equivalent to (2)

$\Leftrightarrow X' \cap \{x \in X \mid \exists y \in Y'. (x, y) \in R\} = \emptyset$

def of $\cap$

$X' \cap \nabla_R(Y') = \emptyset$

def of ∇_R

WTS: $X' \cap \nabla_R(Y') = \emptyset \implies \Delta_R(X') \cap Y' = \emptyset$

$X' \cap \nabla_R(Y') = \emptyset$

suppose

$X' \cap \{x \in X \mid \exists y \in Y'. (x, y) \in R\} = \emptyset$

def of ∇_R

$\{x \in X' \mid \exists y \in Y'. (x, y) \in R\}$

def of $\cap$ (3)

$\forall y \in Y'. \forall x \in X'. (x, y) \notin R$

equivalent to (3) (4)

$\Leftrightarrow \{y \in Y \mid \exists x \in X'. (x, y) \in R\} \cap Y' = \emptyset$

equivalent to (4)

$\Delta_R(X') \cap Y' = \emptyset$

def of Δ_R

A.2 Proposition 3.15

Suppose functions $f : A \rightarrow B$ and $g : B \rightarrow A$ between Boolean algebras. The following statements are equivalent:

(1) f and g form a conjugate pair

(2) f and g° form a Galois connection

PROOF. Fix $x \in A$ and $y \in B$.

WTS: f, g conjugate $\implies f, g^\circ$ form Galois connection.

$f(x) \leq_B y$

(suppose)

$f(x) \wedge_B \neg y = \perp$

$x \wedge_A g(\neg y) = \perp$

(conjugacy)

$x \leq_A \neg(g(\neg y))$

$\Leftrightarrow x \leq_A g^\circ(y)$

(def of $\cdot^\circ$)

WTS: f, g° Galois connection $\implies f, g$ conjugate

$f(x) \wedge_B y = \perp$

suppose

$f(x) \leq_B \neg y$

$x \leq_A g^\circ(\neg y)$

(GC)

$$x \leq_A \neg(g(y)) \quad (\text{def of } \cdot^\circ)$$

$$x \wedge_A g(y) = \perp$$

□

A.3 Lemma 3.13

Fix $R \subseteq X \times Y$. We proceed by direct calculation.

PROOF. **WTS:** $\Delta_R^\circ = \blacktriangle_R$:

$$\Delta_R(X')^\circ = \{y \in Y \mid \exists x \in X'.(x, y) \in R\}^\circ \quad \text{def of } \Delta_R$$

$$= \{y \in Y \mid \exists x \in (X')^c.(x, y) \in R\}^c \quad \text{def of } \cdot^\circ$$

$$= \{y \in Y \mid \exists x \in (X \setminus X').(x, y) \in R\}^c \quad \text{def of } \cdot^c$$

$$= \{y \in Y \mid \nexists x \in (X \setminus X').(x, y) \in R\} \quad \cdot^c \text{ negates predicate}$$

$$= \blacktriangle_R(X') \quad \text{def of } \blacktriangle_R$$

WTS: $\nabla_R^\circ = \blacktriangledown_R$:

$$\nabla_R(Y')^\circ = \{x \in X \mid \exists y \in Y'.(x, y) \in R\}^\circ \quad \text{def of } \nabla_R^\circ$$

$$= \{x \in X \mid \exists y \in (Y')^c.(x, y) \in R\}^c \quad \text{def of } \cdot^\circ$$

$$= \{x \in X \mid \exists y \in (Y \setminus Y').(x, y) \in R\}^c \quad \text{def of } \cdot^c$$

$$= \{x \in X \mid \nexists y \in (Y \setminus Y').(x, y) \in R\} \quad \cdot^c \text{ negates predicate}$$

$$= \blacktriangledown_R(Y') \quad \text{def of } \blacktriangledown_R$$

□

A.4 Proposition 3.17

PROOF.

$\Rightarrow$ **direction:**

$$X \subseteq S(G) \text{ with } X \text{ demBy}_G Y \quad \text{suppose (1)}$$

$$\frac{(X, \emptyset), G \text{ demByV } H}{X \text{ demBy}_G T(H) = Y} \quad \text{inversion; } \exists H$$

$$\Leftrightarrow \Delta_G(X) = T(G) \cap \Delta_G^*(X) = Y \quad (1); \text{A.2; Proposition A.5}$$

$\Leftarrow$ **direction:**

$$X \subseteq S(G) \text{ with } \Delta_G(X) = T(G) \cap \Delta_G^*(X) = Y \quad \text{suppose}$$

$$(X, \emptyset), G \text{ demByV } H \text{ with } T(H) = Y \quad \text{Proposition A.5; } \exists H$$

$$\Leftrightarrow \frac{(X, \emptyset), G \text{ demByV } H}{X \text{ demBy}_G T(H) = Y} \quad \text{def. demBy}$$

□

A.5 Correctness of demByV

The *output* relation of a graph G with reachability relation R , defined as $R \cap V(G) \times T(G)$, gives rise to the following extension of Δ_G :

Definition A.1 (Δ^* for a dependence graph). For a graph G with output relation R , define:

$$\Delta_G^* := \Delta_R : \mathbb{P}(V(G)) \rightarrow \mathbb{P}(V(G))$$

LEMMA A.2. $\Delta_G = T(G) \cap \Delta_G^*(X)$.

LEMMA A.3. If $X \subseteq T(G)$ then $\Delta_G^*(X) = X$.

LEMMA A.4. Suppose $V(H) \subseteq V(G)$. For any $\alpha \in T(H)$ and any $E = \text{outE}_G(\alpha) \neq \emptyset$:

$$\Delta_G^*(T(H)) = \Delta_{G \setminus E}^*(T(H \cup E))$$

PROPOSITION A.5. For $V(H) \subseteq V(G)$, and any $Y \subseteq T(G)$:

$$(\exists H'. H, G \text{ demByV } H' \text{ with } T(H') = Y) \iff T(H \cup G) \cap \Delta_G^*(T(H)) = Y$$

PROOF.

$V(H) \subseteq V(G)$ and $Y \subseteq T(G)$

suppose

$\Rightarrow$ **direction:**

$H, G \text{ demByV } H' \text{ with } T(H') = Y$

suppose; $\exists H'$

By induction on derivation of demByV .

done

Subcase

$$\frac{}{H, G \text{ demByV } H = H'} T(H) \subseteq T(G)$$

$$\Leftrightarrow T(H \cup G) \cap \Delta_G^*(T(H)) = T(H) \cap \Delta_G^*(T(H)) = T(H)$$

Lemma A.3

$$\text{Subcase} \quad \frac{\text{extend} \quad H \cup E, G \setminus E \text{ demByV } H'}{H, G \text{ demByV } H'} \alpha \in T(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$$

$$\Delta_{G \setminus E}^*(T(H \cup E)) = T(H')$$

IH

$$\Leftrightarrow \Delta_G^*(T(H)) = T(H')$$

Lemma A.4

$\Leftarrow$ **direction:**

By strong induction over $|E(G)|$.

$$T(G \cup H) \cap \Delta_G^*(T(H)) = Y$$

suppose (2)

Subcase $T(H) \subseteq T(G)$.

done

$$\frac{}{H, G \text{ demByV } H} T(H) \subseteq T(G)$$

def. demByV

$$\Leftrightarrow T(G \cup H) \cap T(H) = T(G \cup H) \cap \Delta_G^*(T(H)) = Y$$

Lemma A.3; (2)

Subcase $\alpha \in T(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$.

$\exists \alpha, \exists E$

$$E(G \setminus E) \subset E(G)$$

$$H \cup E, G \setminus E \text{ demByV } H' \text{ with } T(H') = T((G \setminus E) \cup (H \cup E)) \cap \Delta_{G \setminus E}^*(T(H \cup E))$$

IH; $\exists H'$

$$T(H') = T(G \cup H) \cap \Delta_G^*(T(H)) = Y$$

Lemma A.4

$$\Leftrightarrow \frac{\text{extend} \quad H \cup E, G \setminus E \text{ demByV } H'}{H, G \text{ demByV } H'} \alpha \in T(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$$

def. demByV

□

A.6 Proposition 3.19

PROOF.

$\Rightarrow$ **direction:**

$X \subseteq S(G)$ and $Y \subseteq T(G)$ with $X \text{ suff}_G Y$

suppose

$$\frac{(X, \emptyset), (\emptyset, \emptyset), G \text{ suffE } H}{X \text{ suff}_G V(H) \cap T(G) = Y}$$

inversion; $\exists H$

$(X, \emptyset), (\emptyset, \emptyset), G$ form a partial slice of G

immediate

$$\Leftrightarrow \blacktriangle_G(X) = T(G) \cap \blacktriangle_G^*(X) = Y$$

Lemma A.10, Proposition A.14

$\Leftarrow$ **direction:**

$X \subseteq S(G)$ and $Y \subseteq T(G)$ with $\blacktriangle_G(X) = Y$

suppose

$$\blacktriangle_G(X) = T(G) \cap \blacktriangle_G^*(X)$$

Lemma A.10

$(X, \emptyset), (\emptyset, \emptyset), G$ form a partial slice of G

immediate

$(X, \emptyset), (\emptyset, \emptyset), G \text{ suffE } H$

Proposition A.14; $\exists H$

$$\Leftrightarrow \frac{(X, \emptyset), (\emptyset, \emptyset), G \text{ suffE } H}{X \text{ suff}_G V(H) \cap T(G) = Y}$$

def. suff

□

A.7 Correctness of suffE

Definition A.6 (Partial Slice). Graphs H, P, G constitute a *partial slice* of the graph G_0 iff H, P, G form a partition of G_0 and:

- (i) $V(H) \subseteq V(P) \subseteq V(G) = V(G_0)$
- (ii) $V(H) \supseteq T(P)^c$
- (iii) $V(H) = S(P)$
- (iv) $V(P) = S(P) \cap T(P)$
- (v) $\forall (\alpha, \beta) \in (S(G_0) \setminus S(H)) \times V(H). (\alpha, \beta) \notin R_{G_0}$
- (vi) $S(G) \setminus (T(P) \setminus S(P)) = V(H) \uplus (S(G_0) \setminus S(H))$

COROLLARY A.7. H, P, G a partial slice $\iff H, P, G$ edge disjoint and $\forall (\alpha, \beta) \in E(G) \cup E(P). \beta \notin V(H)$

Definition A.8 (Input Relation). For a graph G with reachability relation R , define its input relation $R_I := (S(G) \times V(G)) \cap R$

Definition A.9 ($\blacktriangle^$ for a dependence graph).* For a graph G , with an input relation R_I , and a set of input vertices $X \subseteq S(G)$:

$$\blacktriangle_G^*(X) := \blacktriangle_{R_I}(X) = \{y \in V(G) \mid \nexists x \in S(G) \setminus X. (x, y) \in R_I\}$$

LEMMA A.10. $\blacktriangle_G(X) = T(G) \cap \blacktriangle_G^*(X)$

LEMMA A.11. If H, P, G a partial slice of G_0 with $S(G) \cap V(P) \subseteq S(P)$ and $V(H) \subseteq T(G)$:

$$\blacktriangle_{G_0}^*(S(H)) = V(H)$$

PROOF.

H, P, G a partial slice

suppose

$S(G) \cap V(P) \subseteq S(P)$

suppose (1)

$V(H) \subseteq T(G)$

suppose (2)

1520 $\subseteq$ **direction**

1521 $y \in \blacktriangle_{G_0}^*(S(H))$

suppose

1522 $\forall x \in S(G_0) \setminus S(H). (x, y) \notin R_{G_0}$

def. $\blacktriangle^*$

1524 $y \in V(G)$

1525 $x \in S(G). (x, y) \in R_G$

$\exists x, G$ acyclic

1526 $x \in V(H) \cup S(G_0) \setminus S(H) \cup T(P) \setminus S(P)$

H, P, G a partial slice

1527 $x \notin S(G_0) \setminus S(H)$

immediate

1528 $x \notin T(P) \setminus S(P)$

contradicts 1

1529 $\Leftrightarrow x \in V(H)$

$S(G)$ invariant

1530 $x \neq y \implies x \notin T(H) \implies \perp$

contradicts 2

1532 $\Leftrightarrow x = y$

1533 $\Leftrightarrow y \in V(H)$

1534 $\supseteq$ **direction**

1535 $y \in V(H)$

suppose

1536 $y \in V(G_0)$

1537 $S(G_0) \setminus S(H) \subseteq (V(G \cup P) \setminus V(H))$

1538 $\forall \alpha \in S(G_0) \setminus S(H). (\alpha, y) \notin R_{G_0}$

def. partial slice

1540 $\Leftrightarrow y \in \blacktriangle_{G_0}^*(S(H))$

1541

□

1542

1543 **LEMMA A.12.**

1544 (1) If $\alpha \in V(H), E = \text{outE}_G(\alpha) \neq \emptyset$ then:

1545 H, P, G a partial slice of $G_0 \iff H, P \cup E, G \setminus_E E$ a partial slice of G_0

1546 (2) If $\alpha \in S(G), E = \text{inE}_P(\alpha) \neq \emptyset$ then: H, P, G a partial slice $G_0 \iff H \cup E, P \setminus_E E, G$ a partial slice of G_0

1547

1548 **PROOF.**

1549 **(1.II), $\Rightarrow$**

1550 $E = \{\alpha\} \times B$

suppose

1552 $T(P)^c \subseteq V(H)$

suppose

1553 $T(P \cup E)^c = T(P)^c \cup \{\alpha\}$

def. E

1554 $\Leftrightarrow T(P \cup E)^c \subseteq V(H)$

$\alpha \in V(H)$

1555 **(1.II), $\Leftarrow$**

1556 $T(P \cup E)^c \subseteq V(H)$

suppose

1558 $T(P)^c \subseteq T(P \cup E)^c$

1559 $\Leftrightarrow T(P)^c \subseteq V(H)$

1560 **(1.III), $\Rightarrow$**

1561 $E = \{\alpha\} \times B$

suppose

1562 $V(H) = S(P)$

suppose

1564 Need to show: $V(H) = S(P \cup E)$

1565 $S(P \cup E) = S(P) \setminus (S(P) \cap B)$

1566 Need to show: $S(P) \cap B = \emptyset$

1567

1568

1569	$(\alpha, \beta) \in E(G)$	$\forall(\alpha, \beta) \in E$
1570	$\beta \in S(P) = V(H) \implies \perp$	Contra partial slice(V)
1571	$\Leftrightarrow \beta \notin S(G)$	
1572	$\Leftrightarrow S(P) \cap \beta = \emptyset$	
1573	$\Leftrightarrow V(H) = S(P \cup E)$	
1574		
1575	(1.III), $\Leftarrow$	
1576	$V(H) = S(P \cup E)$	suppose
1577	$E = \{\alpha\} \times B$	suppose
1578	Want to show: $V(H) = S(P)$	
1579	$S(P) = S(P \cup E) \setminus B$	
1580	suffices to show: $S(P \cup E) \cap B = \emptyset$	
1581		
1582	$(\alpha, \beta) \in E. (\alpha, \beta) \in E(G)$	
1583	$\Leftrightarrow \beta \notin V(H) = S(P \cup E)$	partial slice (V)
1584	$\Leftrightarrow S(P \cup E) \cap B = \emptyset$	
1585	$\Leftrightarrow S(P) = S(P \cup E)$	
1586	$\Leftrightarrow V(H) = S(P)$	
1587		
1588	(1.IV), $\Rightarrow$	
1589	$V(P) = S(P) \cup T(P)$	suppose
1590	$E = B \times \{\alpha\}$	suppose
1591	$V(P \cup E) = V(P) \cup B$	
1592	suffices to show that: $S(P) \cup T(P) \cup B = S(P \cup E) \cup T(P \cup E)$	
1593	$S(P) = S(P \cup E)$	proved in proof of (iii)
1594	$\alpha \in S(P) = S(P \cup E)$	$\alpha \in V(H)$
1595	$T(P \cup E) = (T(P) \cup B) \setminus \alpha$	def. E
1596		
1597	$\Leftrightarrow S(P) \cup T(P) \cup B = S(P \cup E) \cup ((T(P) \cup B) \setminus \{\alpha\})$	
1598	$\Leftrightarrow S(P) \cup T(P) \cup B = S(P \cup E) \cup T(P \cup E)$	
1599	(1.IV), $\Leftarrow$	
1600	$V(P \cup E) = S(P \cup E) \cup T(P \cup E)$	suppose
1601	$V(P \cup E) = V(P) \cup B$	
1602	$S(P \cup E) \cup T(P \cup E) = V(P) \cup B$	
1603		
1604	$S(P \cup E) = S(P)$	def. E
1605	$T(P \cup E) = (T(P) \cup B) \setminus \{\alpha\}$	def. E
1606	$B \cap S(P) = \emptyset$	
1607	$S(P) \setminus B = S(P)$	
1608	$(S(P \cup E) \cup T(P \cup E)) \setminus B = V(P)$	
1609	$(S(P \cup E) \setminus B) \cup ((T(P) \cup B) \setminus \{\alpha\}) \setminus B = V(P)$	
1610	$\Leftrightarrow S(P) \cup (T(P) \setminus \{\alpha\}) = V(P)$	
1611		
1612	$\Leftrightarrow S(P) \cup T(P) = V(P)$	$\alpha \in S(P)$
1613	(1.V), $\Rightarrow$	
1614	H, P, G a partial slice	suppose
1615	$\forall(\alpha, \beta) \in E(G) \cup E(P). \beta \notin V(H)$	def. partial slice
1616		
1617		

1618	$E(G) \cup E(P) = E(G \setminus_E E) \cup E(P \cup E)$	
1619	$\Leftrightarrow \forall(\alpha, \beta) \in E(G \setminus_E E) \cup E(P \cup E). \beta \notin V(H)$	
1620	(1.V), $\Leftarrow$	
1621	$H, P \cup E, G \setminus_E E$ a partial slice	suppose
1622	$\forall(\alpha, \beta) \in E(G \setminus_E E) \cup E(P \cup E). \beta \notin V(H)$	def. partial slice
1623	$E(G) \cup E(P) = E(G \setminus_E E) \cup E(P \cup E)$	
1624	$\Leftrightarrow \forall(\alpha, \beta) \in E(G) \cup E(P). \beta \notin V(H)$	
1625	(1.VI)	
1626	Suffices to show: $S(G) \setminus (T(P) \setminus S(P)) = S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E))$	
1627	$S(G) \setminus (T(P) \setminus S(P)) = V(H) \uplus (S(G_0) \setminus S(H))$	suppose
1628	$E = \{\alpha\} \times B$	
1629	Subcase $x \in S(G) \setminus (T(P) \setminus S(P))$.	
1630	$x \in S(G) \subseteq S(G \setminus_E E)$	
1631	$x \notin T(P) \setminus S(P)$	(3)
1632	$x \notin B$	$x \in S(G)$
1633	$\Leftrightarrow x \notin T(P \cup E) \setminus S(P \cup E)$	
1634	$x \in S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E))$	
1635	$\Leftrightarrow S(G) \setminus T(P) \setminus S(P) \subseteq S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E))$	(4)
1636	Subcase $x \notin S(G) \setminus (T(P) \setminus S(P))$.	
1637	Subcase $x \in S(G)$.	
1638	$x \in T(P) \setminus S(P)$	(5)
1639	$x \in T(P \cup E) \setminus S(P \cup E)$	
1640	$\Leftrightarrow x \notin S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E))$	
1641	Subcase $x \notin S(G) \wedge x \in S(G \setminus_E E)$.	
1642	$x \notin S(G) \setminus (T(P) \setminus S(P))$	
1643	$(\alpha, x) \in E$	$x \in S(G \setminus_E E) \setminus S(G)$ (6)
1644	$x \in (T(P \cup E) \setminus S(P \cup E))$	
1645	$\Leftrightarrow x \notin S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E))$	
1646	$\Leftrightarrow S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E)) \subseteq S(G) \setminus (T(P) \setminus S(P))$	(7)
1647	$\Leftrightarrow S(G) \setminus T(P) \setminus S(P) = S(G \setminus_E E) \setminus (T(P \cup E) \setminus S(P \cup E))$	
1648	(2.II), $\Rightarrow$	
1649	$E = B \times \{\alpha\}$	suppose
1650	$T(P)^c \subseteq V(H)$	suppose
1651	$V(H \cup E) = V(H) \cup \{\alpha\}$	
1652	$T(P \setminus_E E)^c \cup B = T(P)^c$	
1653	$B \subseteq T(P)^c \subseteq V(H)$	
1654	$\Leftrightarrow T(P \setminus_E E)^c \subseteq V(H) \cup \{\alpha\}$	
1655	(2.II), $\Leftarrow$	
1656	$T(P \setminus_E E)^c \subseteq V(H \cup E)$	suppose
1657	$T(P)^c \subseteq T(P \setminus_E E)^c \cup B$	
1658	$\forall \beta \in B. \beta \in V(H) \cup \{\alpha\}$	def. E
1659		
1660		

1667	$\Leftrightarrow \forall \beta \in B. \beta \in V(H)$	$\beta \neq \alpha$
1668	$\Leftrightarrow T(P \setminus_E E)^c \cup B \subseteq V(H) \cup \{\alpha\}$	
1669	$\Leftrightarrow T(P)^c \subseteq V(H)$	(8)
1670		
1671	(2.III), $\Rightarrow$	
1672	$V(H) = S(P)$	suppose
1673	want to show: $V(H \cup E) = S(P \setminus_E E)$	
1674	$V(H \cup E) = V(H) \cup \{\alpha\} = S(P) \cup \{\alpha\}$	
1675	$S(P \setminus_E E) = S(P) \cup \{\alpha\}$	def. E
1676	$\Leftrightarrow S(P \setminus_E E) = V(H \cup E)$	
1677		
1678	(2.III), $\Leftarrow$	
1679	$V(H \cup E) = S(P \setminus_E E)$	suppose
1680	$V(H) = V(H \cup E) \setminus \{\alpha\}$	
1681	$S(P \setminus_E E) = S(P) \cup \{\alpha\}$	def. E
1682	$\Leftrightarrow S(P \setminus_E E) \setminus \{\alpha\} = S(P)$	
1683	$\Leftrightarrow V(H) = S(P)$	
1684		
1685	(2.IV), $\Rightarrow$	
1686	$V(P) = S(P) \cup T(P)$	suppose (9)
1687	want to show: $V(P) = S(P \setminus_E E) \cup T(P \setminus_E E)$	
1688	$S(P) \subseteq S(P \setminus_E E) \wedge T(P) \subseteq T(P \setminus_E E)$	straightforward
1689	Remains to show: $S(P \setminus_E E) \cup T(P \setminus_E E) \subseteq S(P) \cup T(P)$	
1690	$x \in V(P \setminus_E E)$	suppose
1691	Subcase $x \in S(P \setminus_E E)$.	
1692	$x \in V(P)$	
1693		
1694	$\Leftrightarrow x \in S(P) \cup T(P)$	9 (10)
1695	Subcase $x \in T(P \setminus_E E)$.	
1696	$x \in V(P)$	
1697	$\Leftrightarrow x \in S(P) \cup T(P)$	9
1698		
1699	(2.IV), $\Leftarrow$	
1700	$V(P) = V(P \setminus_E E) = S(P \setminus_E E) \cup T(P \setminus_E E)$	suppose
1701	want to show: $S(P \setminus_E E) \cup T(P \setminus_E E) = S(P) \cup T(P)$	
1702	$S(P) \cup T(P) \subseteq S(P \setminus_E E) \cup T(P \setminus_E E)$	
1703	remains to show: $S(P \setminus_E E) \cup T(P \setminus_E E) \subseteq S(P) \cup T(P)$	
1704	$x \in V(P)$	suppose
1705	Subcase $x \in S(P \setminus_E E)$.	
1706	$S(P \setminus_E E) \setminus S(P) = \{\alpha\}$	def. E
1707		
1708	Subcase $x = \alpha$.	
1709	$\alpha \notin T(P)$	assume
1710	$\alpha \in V(H)$	partial slice (II)
1711	$(\beta, \alpha) \in E(P)$	def. E
1712	$\Leftrightarrow \perp$	contra partial slice (V) (11)
1713	$\Leftrightarrow \alpha \in T(P)$	
1714		
1715		

1716	$\Leftrightarrow x \in S(P) \cup T(P)$	
1717	Subcase $x \in S(P)$.	
1718	$\Leftrightarrow x \in S(P) \cup T(P)$	
1719	Subcase $x \in T(P \setminus_E E)$.	
1720	$T(P \setminus_E E) \setminus T(P) \subseteq B$	def. E
1721		
1722	Subcase $x \in B$.	
1723	$B \subseteq V(H \cup E)$	def. E (12)
1724	$\alpha \notin B$	
1725	$\Leftrightarrow B \in S(P)$	
1726	$\Leftrightarrow x \in S(P) \subseteq S(P) \cup T(P)$	
1727	Subcase $x \in T(P)$.	
1728	$\Leftrightarrow x \in S(P) \cup T(P)$	
1729		
1730	$\Leftrightarrow T(P \setminus_E E) \subseteq S(P) \cup T(P)$	(13)
1731	(2.V), $\Rightarrow$	
1732	H, P, G a partial slice	suppose
1733	$\forall \beta \in V(G). \nexists (\beta, \alpha) \in E(G)$	$\alpha \in S(G)$ (14)
1734	$\forall (x, y) \in E(G). y \notin V(H \cup E)$	def. partial slice, 14
1735		
1736	$V(H \cup E) = V(H) \cup \{\alpha\}$	def. E
1737	$(x, y) \in E(P)$	suppose
1738	Subcase $(x, y) \in E(P) \setminus_E E$.	
1739	$y \notin V(H)$	
1740	$y \neq \alpha$	$(x, y) \notin E$
1741	$\Leftrightarrow y \notin V(H \cup E)$	
1742		
1743	Subcase $(x, y) \in E$.	
1744	$\Leftrightarrow (x, y) \notin E(P) \setminus_E E$	
1745	(2.V), $\Leftarrow$	
1746	$H \cup E, P \setminus_E E, G$ a partial slice	suppose
1747	$\forall (\alpha, \beta) \in E(G) \cup E(P \setminus_E E). \beta \notin V(H \cup E)$	def. partial slice
1748	$\forall (\alpha, \beta) \in E(G) \cup E(P \setminus_E E). \beta \notin V(H)$	immediate
1749		
1750	$(\beta, \alpha) \in E \subset E(P \setminus_E E) \cup E = E(P)$	suppose
1751	$\alpha \notin V(H)$	def. E
1752	$\Leftrightarrow \forall (\alpha', \beta') \in E(P)$	
1753	$\Leftrightarrow \forall (\alpha', \beta') \in E(G) \cup E(P). \beta \notin V(H)$	
1754		
1755	(2.VI)	
1756	Suffices to show: $T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\} = T(P) \setminus S(P)$	
1757	$E = \text{in}_P(\alpha) \times \{\alpha\}$	
1758	Subcase $T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\} \subseteq T(P) \setminus S(P)$.	
1759	Subcase $x = \alpha$.	
1760	$\alpha \notin S(P)$	$E \neq \emptyset$
1761	$\alpha \in T(P)$	P has depth 1
1762	$\Leftrightarrow x \in T(P) \setminus S(P)$	
1763		
1764		

Subcase $x \neq \alpha \wedge x \in T(P \setminus_E E) \setminus S(P \setminus_E E)$.
 $T(P \setminus_E E) \setminus T(P) \subseteq \text{in}_P(\alpha)$
 $\text{in}_P(\alpha) \subseteq S(P) \subseteq S(P \setminus_E E)$
 $\Leftrightarrow x \in T(P)$ $x \in T(P \setminus_E E) \wedge \notin \text{in}_P(\alpha)$
 $S(P \setminus_E E) \supseteq S(P)$
 $x \notin S(P)$ $x \notin S(P \setminus_E E)$
 $\Leftrightarrow x \in T(P) \setminus S(P)$
 $\Leftrightarrow T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\} \subseteq T(P) \setminus S(P)$
Subcase $T(P) \setminus S(P) \subseteq T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\}$.
 $x \in T(P) \setminus S(P)$ suppose
 $x \notin S(P)$ (15)
 $x \in T(P) \subseteq T(P \setminus_E E)$
Subcase $x \notin S(P \setminus_E E)$.
 $\Leftrightarrow x \in T(P \setminus_E E) \setminus S(P \setminus_E E) \subseteq T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\}$
Subcase $x \in S(P \setminus_E E)$.
 $x \in S(P \setminus_E E) \setminus S(P)$
 $S(P \setminus_E E) \setminus S(P) = \{\alpha\}$
 $\Leftrightarrow x = \alpha \in T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\}$
 $\Leftrightarrow T(P) \setminus S(P) \subseteq T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\}$
 $\Leftrightarrow T(P) \setminus S(P) = T(P \setminus_E E) \setminus S(P \setminus_E E) \cup \{\alpha\}$
□
LEMMA A.13. *Suppose H, P, G a partial slice of G_0 , then:*
(1) $\blacktriangle_{G \cup P \cup H}^*(S(H)) = \blacktriangle_{(G \setminus_E E) \cup (P \cup E) \cup H}^*(S(H))$
(2) $\blacktriangle_{G \cup P \cup H}^*(S(H)) = \blacktriangle_{G \cup (P \setminus_E E) \cup (H \cup E)}^*(S(H \cup E))$
PROOF. (1) $G_0 = G \cup P \cup H = (G \setminus_E E) \cup (P \cup E) \cup H$
(2) $G_0 = G \cup P \cup H = G \cup (P \setminus_E E) \cup (H \cup E)$, and $S(H) = S(H \cup E)$
□
PROPOSITION A.14. *Suppose H, P, G a partial slice of G_0 :*
 $(\exists H'. H, P, G \text{ suffE } H' \text{ with } V(H') = Y) \iff \blacktriangle_{G_0}^*(S(H)) = Y$
PROOF.
 H, P, G a partial slice suppose (16)
 $\Rightarrow$ **direction:**
 $H, P, G \text{ suffE } H' \text{ with } V(H') = Y$ suppose, $\exists H'$
By induction on derivation of suffE.
done
Subcase

$$\frac{H, P, G \text{ suffE } H}{S(G) \cap V(P) \subseteq S(P) \wedge V(H) \subseteq T(G)}$$

 $\Leftrightarrow \blacktriangle_{G_0}^*(S(H)) = V(H)$ Lemma A.11

1814		pending	
1815	Subcase	$\frac{H, P \cup E, G \setminus_E E \text{ suffE } H'}{H, P, G \text{ suffE } H'} \alpha \in V(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$	
1816			
1817	$H, P \cup E, G \setminus_E E$ a partial slice of G_0		Lemma A.12
1818	$\mathbf{\Delta}_{(G \setminus_E E) \cup (P \cup E) \cup H}^*(S(H)) = Y$		IH
1819	$\Leftrightarrow \mathbf{\Delta}_{G_0}^*(S(H)) = Y$		Lemma A.13(i)
1820			
1821			
1822	Subcase	$\frac{\text{extend } H \cup E, P \setminus_E E, G \text{ suffE } H'}{H, P, G \text{ suffE } H'} \alpha \in S(G) \wedge E = \text{inE}_P(\alpha) \neq \emptyset$	
1823			
1824	$H \cup E, P \setminus_E E, G$ a partial slice of G_0		Lemma A.12
1825	$\mathbf{\Delta}_{G \cup (P \setminus_E E) \cup (H \cup E)}^*(S(H \cup E))$		IH
1826	$\Leftrightarrow \mathbf{\Delta}_{G_0}^*(S(H))$		Lemma A.13(ii)
1827	$\Leftrightarrow \mathbf{\Delta}_{G_0}^*(S(H))$		
1828	$\Leftarrow$ direction		
1829	By strong lexicographical induction over $(E(G) , E(P))$		
1830	$\mathbf{\Delta}_{G_0}^*(S(H)) = Y$		suppose
1831	Subcase $S(G) \cap V(P) \subseteq S(P) \wedge V(H) \subseteq T(G)$.		
1832			
1833	done		
1834			def. suffE
1835	$\frac{}{H, P, G \text{ suffE } H} S(G) \cap V(P) \subseteq S(P) \wedge V(H) \subseteq T(G)$		
1836			
1837	$\Leftrightarrow \mathbf{\Delta}_{G_0}^*(S(H)) = V(H) = Y$		Lemma A.11
1838	Subcase $\alpha \in V(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$.		$\exists \alpha, \exists E$
1839	$(E(G \setminus_E E) , E(P)) < (E(G) , E(P \cup E))$		
1840	$H, P \cup E, G \setminus_E E$ a partial slice of G_0		Lemma A.12
1841	$H, P \cup E, G \setminus_E E \text{ suffE } H'$		IH; $\exists H'$
1842			
1843	pending		
1844	$\frac{H, P \cup E, G \setminus_E E \text{ suffE } H'}{H, P, G \text{ suffE } H'} \alpha \in V(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$		def. suffE
1845			
1846	$\mathbf{\Delta}_{(G \setminus_E E) \cup (P \cup E) \cup H}^*(S(H)) = \mathbf{\Delta}_{G \cup P \cup H}^*(S(H))$		Lemma A.13(i)
1847	$\Leftrightarrow \mathbf{\Delta}_{G_0}^*(S(H)) = V(H') = Y$		IH
1848	Subcase $\alpha \in S(G) \wedge E = \text{inE}_P(\alpha) \neq \emptyset$.		$\exists \alpha, \exists E$
1849	$(E(G) , E(P \setminus_E E)) < (E(P) , E(P))$		
1850	$H \cup E, P \setminus_E E, G$ a partial slice of G_0		Lemma A.12
1851	$H \cup E, P \setminus_E E, G \text{ suffE } H'$		IH, $\exists H'$
1852			
1853	extend		
1854	$\frac{H \cup E, P \setminus_E E, G \text{ suffE } H'}{H, P, G \text{ suffE } H'} \alpha \in S(G) \wedge E = \text{inE}_P(\alpha) \neq \emptyset$		def. suffE
1855			
1856	$\mathbf{\Delta}_{G \cup P \cup H}^*(S(H)) = \mathbf{\Delta}_{G \cup (P \setminus_E E) \cup (H \cup E)}^*(S(H \cup E))$		Lemma A.13(ii)
1857	$\Leftrightarrow \mathbf{\Delta}_{G_0}^*(S(H)) = V(H') = Y$		(17)
1858			IH
1859			
1860			$\square$
1861			
1862			

B SOURCE CODE FOR DATA VISUALISATION FIGURES AND BENCHMARKED PROGRAMS

```

1863 1 let isCountry name x = name == x.country;
1864 2   isYear year x = year == x.year;
1865 3
1866 4 let plot year countries =
1867 5   let rens = filter (isYear year) renewables;
1868 6   nonRens = filter (isYear year) nonRenewables;
1869 7   let plotCountry country =
1870 8     let rens' = filter (isCountry country) rens;
1871 9     rensOut = sum (map (fun x → x.output) rens');
187210     rensCap = sum (map (fun x → x.capacity) rens');
187311     x = head (filter (isCountry country) nonRens);
187412     nonRensCap = x.nuclearCap + x.petrolCap + x.gasCap + x.coalCap
187513   in {
187614     x: rensCap / (rensCap + nonRensCap),
187715     y: (rensOut + x.nuclearOut) / (rensCap + x.nuclearCap)
187816   }
187917 in map plotCountry countries
188018
188119 in ScatterPlot {
188220   caption: "Clean energy efficiency vs. proportion of renewable energy capacity",
188321   data: plot 2018 [ "BRA", "CHN", "DEU", "FRA", "EGY", "IND", "JPN", "MEX", "NGA", "USA" ],
188422   xlabel: "Renewables/TotalEnergyCap",
188523   ylabel: "Clean Capacity Factor"
188624 }

```

Fig. 14. Source code: energy-query (Figure 2 that visualises linked inputs)

```

1912 1 MultiPlot {
1913 2   "stacked-bar-chart" :=
1914 3     let totalFor year country rows =
1915 4       let [ row ] = [ row | row ← rows, row.year == year, row.country == country ]
1916 5       in row.nuclearOut + row.gasOut + row.coalOut + row.petrolOut;
1917 6     let stack year = [ { y: country, z: totalFor year country nonRenewables }
1918 7       | country ← [ "BRA", "EGY", "IND", "JPN" ] ]
1919 8     in BarChart {
1920 9       caption: "Non-renewables by country",
1921 10      data: [ { x: numToStr year, bars: stack year }
1922 11        | year ← [2014..2018] ]
1923 12    },
1924 13   "scatter-plot" :=
1925 14     let isCountry name x = name == x.country;
1926 15     let isYear year x = year == x.year;
1927 16
1928 17     let plot year countries =
1929 18       let rens = filter (isYear year) renewables;
1930 19       nonRens = filter (isYear year) nonRenewables;
1931 20       let plotCountry country =
1932 21         let rens' = filter (isCountry country) rens;
1933 22         rensOut = sum (map (fun x → x.output) rens');
1934 23         rensCap = sum (map (fun x → x.capacity) rens');
1935 24         x = head (filter (isCountry country) nonRens);
1936 25         nonRensCap = x.nuclearCap + x.petrolCap + x.gasCap + x.coalCap
1937 26       in {
1938 27         x: rensCap / (rensCap + nonRensCap),
1939 28         y: (rensOut + x.nuclearOut) / (rensCap + x.nuclearCap)
1940 29       }
1941 30     in map plotCountry countries
1942 31
1943 32   in ScatterPlot {
1944 33     caption: "Clean energy efficiency vs proportion of renewable energy capacity",
1945 34     data: plot 2018 [ "BRA", "CHN", "DEU", "FRA", "EGY"
1946 35       , "IND", "JPN", "MEX", "NGA", "USA" ],
1947 36     xlabel: "Renewables/TotalEnergyCap",
1948 37     ylabel: "Clean Capacity Factor"
1949 38   }
1950 39 }

```

Fig. 15. Source code: stacked-bar-scatter-plot (Figure 3 that visualises linked outputs)

```

1961 1 let filter =
1962 2     let edgeDetect = [[0, 1, 0],
1963 3                       [1, -4, 1],
1964 4                       [0, 1, 0]] in
1965 5     [[ nth2 i j edgeDetect | (i, j) in (3, 3) ]];
1966 6 convolve input_image filter extend
1967
1968                                     (a) edge-detect
1969
1969 1 let filter =
1970 2     let emboss = [[-2, -1, 0],
1971 3                  [-1, 1, 1],
1972 4                  [0, 1, 2]] in
1973 5     [[ nth2 i j emboss | (i, j) in (3, 3) ]];
1974 6 convolve input_image filter zero
1975
1976                                     (b) emboss
1977
1976 1 let filter =
1977 2     let gaussian = [[1, 4, 1],
1978 3                    [4, 16, 4],
1979 4                    [1, 4, 1]] in
1980 5     [[ nth2 i j gaussian | (i, j) in (3, 3) ]];
1981 6 convolve input_image filter zero
1982
1983                                     (c) gaussian
1984
1983 1 let zero n = const n;
1984 2     extend n = min (max n 1);
1985 3 let convolve image kernel method =
1986 4     let ((m, n), (i, j)) = (dims image, dims kernel);
1987 5     (half_i, half_j) = (i `quot` 2, j `quot` 2);
1988 6     area = i * j
1989 7     in [[ let weightedSum = sum [
1990 8           image!(x, y) * kernel!(i' + 1, j' + 1)
1991 9           | (i', j') ← range (0, 0) (i - 1, j - 1),
1992 10          let x = method (m' + i' - half_i) m,
1993 11          let y = method (n' + j' - half_j) n,
1994 12          x >= 1, x <= m, y >= 1, y <= n
1995 13          ] in weightedSum `quot` area
1996 14          | (m', n') in (m, n) ]];
1997 15 let input_image =
1998 16     let image = [[15, 13, 6, 9, 16],
1999 17                  [12, 5, 15, 4, 13],
2000 18                  [14, 9, 20, 8, 1],
2001 19                  [4, 10, 3, 7, 19],
2002 20                  [3, 11, 15, 2, 9]] in
2003 21     [[ nth2 i j image | (i, j) in (5, 5) ]];
2004
2005                                     (d) Library code

```

Fig. 16. Source code: convolution examples

```

2010 1  let year = 2015;
2011 2  let exclude countries yearData =
2012 3      flip map yearData
2013 4          (second (filter (fun (country, countryData) → not (elem country countries))))
2014 5  in caption ("Renewables (GW) by country and energy type, " ++ numToStr year)
2015 6      (groupedBarChart True colours1 0.2
2016 7          (exclude [] (lookup year data)))

```

(a) grouped-bar-chart

```

2018 1  let year1 = head data;
2019 2  let addTotal kvs =
2020 3      ("Total", sum (map snd kvs)) : kvs;
2021 4  let countryData country =
2022 5      map (second (compose addTotal (lookup country))) data in
2023 6  (fun country →
2024 7      caption country
2025 8          (lineChart True ("black" : colours1) (fst year1)
2026 9              (countryData country)))
2027 10 "China"

```

(b) line-chart

```

2030 1  let year = 2015 in
2031 2  caption ("Total renewables (GW) by country, " ++ numToStr year)
2032 3      (stackedBarChart True colours1 0.2 (lookup year data))

```

(c) stacked-bar-chart

Fig. 17. Source code: graphics examples

```

2059 1  let coords (Group gs) = Point 0 0;
2060 2      coords (Rect x y _ _ _) = Point x y;
2061 3      coords (Text x y _ _ _) = Point x y;
2062 4      coords (Viewport x y _ _ _ _ _ _) = Point x y;
2063 5  let get_x g = let Point x _ = coords g in x;
2064 6  let get_y g = let Point _ y = coords g in y;
2065 7  let set_x x (Group gs) = error "Group has immutable coordinates";
2066 8      set_x x (Rect _ y w h fill) = Rect x y w h fill;
2067 9      set_x x (Text _ y str anchor baseline) = Text x y str anchor baseline;
2068 10      set_x x (Viewport _ y w h fill margin scale translate g) =
2069 11          Viewport x y w h fill margin scale translate g;
2070 12 let dimensions2 (Point x1 y1, Point x2 y2) = Point (max x1 x2) (max y1 y2);
2071 13 let dimensions (Group gs) = foldl (curry dimensions2) (Point 0 0) (map (coords_op) gs);
2072 14 dimensions (Polyline ps _ _) = foldl (curry dimensions2) (Point 0 0) ps;
2073 15 dimensions (Rect _ _ w h _) = Point w h;
2074 16 dimensions (Text _ _ _ _ _) = Point 0 0;
2075 17 dimensions (Viewport _ _ w h _ _ _ _) = Point w h;
2076 18 let coords_op g =
2077 19     let (Point x y, Point w h) = prod coords dimensions g in
2078 20     Point (x + w) (y + h);
2079 21 let width g = let Point w _ = dimensions g in w;
2080 22 let height g = let Point _ h = dimensions g in h;
2081 23 let spaceRight z sep gs =
2082 24     zipWith set_x (iterate (length gs) ((+) sep) z) gs;
2083 25 let colours1 = [ "#66c2a5", "#a6d854", "#ffd92f", "#e5c494"
2084 26     , "#fc8d62", "#b3b3b3", "#8da0cb", "#e78ac3"];
2085 27 let colours2 = [ "#e41alc", "#377eb8", "#4daf4a", "#984ea3"
2086 28     , "#ff7f00", "#ffff33", "#a65628", "#f781bf"];
2087 29 let scaleToWidth w (Rect x y _ h fill) = Rect x y w h fill;
2088 30 scaleToWidth w (Viewport x y w0 h fill margin (Scale x_scale y_scale) translate g) =
2089 31     let scale = Scale (x_scale * w / w0) y_scale in
2090 32     Viewport x y w h fill margin scale translate g;
2091 33 let stackRight sep gs =
2092 34     map (scaleToWidth (1 - sep)) (spaceRight (sep / 2) 1 gs);
2093 35 let groupRight sep gs =
2094 36     Viewport 0 0 (length gs) (maximum (map height gs)) "none" 0 (Scale 1 1)
2095 37     (Translate 0 0) (Group (stackRight sep gs));
2096 38 let tickEvery n =
2097 39     let m = floor (logBase 10 n) in
2098 40     if n <= 2 * 10 ** m then 2 * 10 ** (m - 1) else 10 ** m;

```

Fig. 17. Source code: library code for graphics examples (1 of 3)


```

2108 1  let axisStrokeWidth = 0.5;
2109 2      axisColour = "black";
2110 3      backgroundColour = "white";
2111 4      defaultMargin = 24;
2112 5      markerRadius = 3.5;
2113 6      tickLength = 4;
2114 7  let tick Horiz colour len = Line (Point 0 0) (Point 0 (0 - len)) colour axisStrokeWidth;
2115 8      tick Vert colour len = Line (Point 0 0) (Point (0 - len) 0) colour axisStrokeWidth;
2116 9  let label Horiz x distance str = Text x (0 - distance - 4) str "middle" "hanging";
2117 10     label Vert x distance str = Text (0 - distance) x str "end" "central";
2118 11  let labelledTick orient colour len str =
2119 12     Group [tick orient colour len, label orient 0 len str];
2120 13  let mkPoint Horiz x y = Point y x;
2121 14     mkPoint Vert x y = Point x y;
2122 15  let axis orient x start end =
2123 16     let tickSp = tickEvery (end - start);
2124 17     firstTick = ceilingToNearest start tickSp;
2125 18     lastTick = ceilingToNearest end tickSp;
2126 19     n = floor ((end - firstTick) / tickSp) + 1;
2127 20     ys = iterate n ((+) tickSp) firstTick;
2128 21     ys = if firstTick > start then start : ys else ys;
2129 22     ys = if lastTick > end then concat2 ys [lastTick] else ys;
2130 23     ps = map (mkPoint orient x) ys;
2131 24     ax = Group [
2132 25         Line (head ps) (last ps) axisColour axisStrokeWidth,
2133 26         Polymarkers ps (flip map ys (compose (labelledTick orient axisColour tickLength)
2134 27             numToStr))
2135 28     ]
2136 29     in (ax, lastTick);
2137 30  let catAxis orient x catValues =
2138 31     let ys = iterate (length catValues + 1) ((+) 1) 0;
2139 32     ps = map (mkPoint orient x) ys
2140 33     in Group [
2141 34         Line (head ps) (last ps) axisColour axisStrokeWidth,
2142 35         Polymarkers (tail ps) (map (const (tick orient axisColour tickLength)) catValues),
2143 36         Polymarkers (flip map (tail ps) (fun (Point x y) → Point (x - 0.5) y))
2144 37             (map (label orient -0.5 0) catValues)
2145 38     ];

```

Fig. 17. Source code: library code for graphics examples (2 of 3)

```

2157 1 let viewport x_start x_finish y_finish margin gs =
2158 2   Viewport 0 0 (x_finish - x_start) y_finish backgroundColour margin
2159 3     (Scale 1 1) (Translate (0 - x_start) 0) (Group gs);
2160 4 let lineChart withAxes colours x_start data =
2161 5   let xs = map fst data;
2162 6   nCat = length (snd (head data));
2163 7   let plot (n, colour) =
2164 8     let ps = map (fun (x, kvs) → Point x (snd (nth n kvs))) data
2165 9     in Group [
2166 10      Polyline ps colour 1,
2167 11      Polymarkers ps (repeat (length ps) (Circle 0 0 markerRadius colour))
2168 12    ];
2169 13 let lines = zipWith (curry plot) (iterate nCat ((+ 1) 0) colours;
2170 14   x_finish = last xs;
2171 15   y_finish = maximum (flip map data (fun (_, kvs) → maximum (map snd kvs)))
2172 16 in if withAxes
2173 17 then
2174 18   let (x_axis, x_finish) = axis Horiz 0 x_start x_finish;
2175 19   (y_axis, y_finish) = axis Vert x_start 0 y_finish
2176 20   in viewport x_start x_finish y_finish' defaultMargin (x_axis : y_axis : lines)
2177 21 else viewport x_start x_finish y_finish 0 lines;
2178 22 let categoricalChart plotValue withAxes colours sep data =
2179 23 let gs = stackRight sep (plotValue colours (map snd data));
2180 24 w = length gs;
2181 25 h = maximum (map height gs)
2182 26 in if withAxes
2183 27 then
2184 28   let x_axis = catAxis Horiz 0 (map fst data);
2185 29   (y_axis, h') = axis Vert 0 0 h
2186 30   in viewport 0 w h' defaultMargin (concat2 gs [x_axis, y_axis])
2187 31 else viewport 0 w h 0 gs;
2188 32 let rects colours ns = zipWith (fun colour n → Rect 0 0 1 n colour) colours ns;
2189 33 let stackedBar colours ns =
2190 34 let heights = map snd ns;
2191 35 subtotals = scanl1 (+) 0 heights;
2192 36 dims = zip (0 : subtotals) heights;
2193 37 rects = map
2194 38   (fun ((y, height), colour) → Rect 0 y 1 height colour)
2195 39   (zip dims colours)
2196 40 in Viewport 0 0 1 (last subtotals) "none" 0 (Scale 1 1) (Translate 0 0) (Group rects);
2197 41 let barChart = categoricalChart rects;
2198 42 let groupedBarChart = categoricalChart (compose map (flip (barChart False) 0));
2199 43 let stackedBarChart = categoricalChart (compose map stackedBar);
2200 44 let caption str (Viewport x y w h fill margin scale translate g) =
2201 45 let g' = Group [ Text (x + w / 2) -2 str "middle" "hanging",
2202 46   Viewport 0 0 w h fill margin scale translate g ]
2203 47 in Viewport x y w h backgroundColour (defaultMargin / 2 + 4) (Scale 1 1) (Translate 0 0) g';
2204
2205

```

Fig. 17. Source code: library code for graphics examples (3 of 3)